Relatório - João Felipe

1. O que foi feito

Implementei as 7 rotas obrigatórias do enunciado (cadastro, listagem com filtros, busca por id, edição parcial, remoção, venda e valor total do estoque) com todas as regras de negócio e de validação. Também fiz os quatro bônus: busca por nome sem diferenciar maiúsculas, ordenação por preço ou nome, a rota `GET /plantas/esgotadas` e o script de seed.

Além do pedido, o projeto tem:

- 51 testes automatizados (Vitest e Supertest), que cobrem a tabela de testes do enunciado, a venda simultânea e alguns casos de borda;
- um painel web em `/loja`, que consome a própria API;
- a documentação da API no `README.md`.

Para conferir o resultado, clonei o repositório do GitHub numa pasta nova e segui o meu próprio README do zero: `docker compose up`, `yarn migration`, `yarn typecheck`, `yarn test` e `yarn seed`. Tudo passou. Também abri o painel no navegador.

2. Como foi feito

Segui a ordem do enunciado: model e migration, repository, controller, rotas, filtros, venda e valor total. Depois vieram os bônus, os testes e o painel. Usei o CRUD de usuário como modelo de organização e de estilo, e fiz um commit por etapa.

As decisões principais:

- **Uma única função de validação para cadastro e edição.** No cadastro todos os campos são obrigatórios, e na edição nenhum é, mas os enviados seguem as mesmas regras. Um parâmetro `parcial` resolve os dois casos sem repetir regras.
- **Presença checada com `=== undefined`, e não com `!campo`.** `!0` e `!false` são verdadeiros, então `!campo` recusaria uma planta com estoque zero ou que não é pet-friendly, as duas válidas.
- **Venda protegida contra duas compras simultâneas.** Se eu buscasse a planta, conferisse o estoque e só depois diminuísse, duas compras do último vaso poderiam passar pela conferência ao mesmo tempo, e o estoque ficaria negativo. Coloquei a condição `estoque >= quantidade` dentro do próprio `updateMany`, então o banco confere e diminui numa única operação. Se nenhuma linha for alterada, a resposta é 400. Um teste manda duas vendas do último vaso ao mesmo tempo e confere que só uma passa.
- **Buscar a planta antes de editar, deletar ou vender**, para devolver 404. Sem isso, o erro do Prisma cairia no `catch` e viraria um 500 errado.
- **Rotas fixas antes de `/plantas/:id`.** Senão `/plantas/esgotadas` seria tratada como um id.
- **Um único `where` para todos os filtros**, porque o Prisma ignora condições com valor `undefined`.

Decisões que o enunciado não cobre:

- `PATCH` sem nenhum campo da planta responde 400, em vez de um 200 que não muda nada;
- campos desconhecidos no corpo (como `id`) são ignorados;
- números acima de 2.147.483.647, o limite do `Int` do PostgreSQL, respondem 400. Sem isso, o banco recusaria o valor e a resposta viraria 500;
- JSON malformado no corpo responde 400 em JSON, e não a página HTML padrão do Express.

3. Principais dificuldades

Nenhuma foi no código da API. Todas foram para fazer o ambiente rodar na minha máquina:

- **Porta 5432 ocupada.** Já existe um Postgres instalado no meu Ubuntu usando essa porta. O container do banco não subia (`address already in use`) e o `yarn migration` falhava com `P1000: Authentication failed`, porque ele estava falando com o Postgres do sistema, e não com o do Docker. Entendi a causa ao ler a mensagem do `docker compose up`. Resolvi trocando `DATABASE_PORT` para 5433 no `.env`, sem mexer no Postgres do sistema.
- **Banco ainda iniciando.** No clone limpo, o `yarn migration` rodou logo depois do `docker compose up` e deu `P1001: Can't reach database server`. O Postgres leva alguns segundos para ficar pronto com o volume novo. Depois de um tempo, esperei e repeti, e funcionou.

Registrei as duas últimas na tabela "Problemas comuns" do README.

O que aprendi: a mensagem de erro diz onde olhar. `P1000` quer dizer que alcancei um servidor que recusou a senha, e `P1001` quer dizer que nem alcancei.

4. Uso de IA

Usei o Claude para:

- planejar as etapas do desafio a partir do enunciado
- escrever maior parte do código do repository, do controller, das rotas, dos testes, do seed e do painel web.
- ajudar a estruturar o README e o rascunho do relatório
- me guiar passo a passo para rodar o projeto e para diagnosticar os erros.

O que fiz eu mesmo: fui revisando o código, rodei tudo na minha máquina, resolvi os problemas de ambiente, criei a migration com `yarn migration`, fiz os commits, testei o projeto num clone limpo e subi o repositório.